

Evidence-Bound Recovery for Ambiguous Submissions in Local Generative-AI Pipelines

Tool Description

Anonymous submission draft

Abstract—When a generation backend accepts a request but its response is lost, retrying can submit an operation whose backend work has already completed. We present a tool that preserves the unresolved client record, exports its interpretation alongside backend evidence, and guards further submission through the same run. A six-operation Windows/ComfyUI paired campaign compares guarded and unguarded products under acceptance-response loss and a fault before upstream sending. The guard withheld the second submission in both conditions, leaving the original run unresolved. The unguarded product retried and completed. Under response loss, it recorded two backend lifecycles rather than one, but the second reused the generation graph’s cache. Synthetic probes also exercised completion/recovery separation. For pipeline integrators, the tool makes the submission/availability tradeoff inspectable without treating backend completion as client recovery.

I. INTRODUCTION

An image-generation client can lose a response after its backend accepts the workflow. Without the returned job identifier, the client cannot determine whether resubmission will repeat completed work. Request counts and timeouts alone leave that question unanswered. Recovery therefore requires an explicit account of the evidence available to the client and the backend.

Remote procedure call semantics already address uncertainty under communication failure [1]. Idempotent APIs make retries safe through request identifiers and an explicit service contract [2]. In the evaluated local generation workflow, backend deduplication is unestablished. Losing the acceptance response can leave a persistent client run without a job identifier even when a separate observer binds backend completion. The recovery decision must account for both records.

We contribute a tool that preserves this disagreement for inspection and constrains the next submission through the original run. Its read-only exporter keeps reported state alongside a versioned interpretation with mapping reasons and supplied backend observations. The guard consults the unresolved attempt before permitting further generation. Synthetic CPU fixtures and controlled Windows campaigns exercise these interfaces, including a case in which backend completion coexists with an incomplete client operation. Section V places this evidence-handling design beside existing retry and tracing contracts.

A paired same-run evaluation further tests the submission boundary against an unguarded product under two transport faults. Its result is a concrete tradeoff: the guard suppresses

another submission, including when the first request never reached the backend, while the original run remains unresolved.

II. FAILURE MODEL AND EVIDENCE SEMANTICS

A **product call** is an invocation by the research client. A **run** is the persistent product object created by `local_gpu_start_run`. A **submission** in the Windows results is a proxy-observed `POST /prompt`; a call that fails before that boundary need not produce a submission. An **execution instance** is an observation bound to backend execution events. CPU fixtures instead observe the synthetic worker entry. These units are not interchangeable.

F00 is a normal control. F02 suppresses the first accepted submission response, creating a window in which the client lacks an acknowledged job identifier. F03, covered by the CPU fixtures rather than the Windows campaign reported here, concerns loss of completion information after a job identifier is known. An explicit rejection and a missing response have different evidential meanings. Neither should be silently converted into a backend execution outcome.

The evidence model distinguishes reported state, interpreted state and oracle state. In addition, the Windows research client emits its own call-level labels. Its `resolved` label is derived from a generated product result after checking for artifact hashes. Its `unresolved` label can arise from several client errors. A research-client label alone does not establish a durable product manifest’s recovery state. In F02, the proxy’s accepted-response loss and the oracle binding provide the additional context.

III. TOOL DESIGN

A. Evidence-preserving export

The exporter accepts a record or run manifest and produces a separate normalized record. It copies reported state, records the mapping version and reasons, and retains supplied oracle state separately. The schema includes source identity, request and input digests, backend identity, job identifier, artifact hash and validator version. Missing fields carry explicit reasons instead of fabricated values. The inspected mapping is `research-normalization-v4`.

Interpretation checks more than an execution-success label. Job and artifact bindings, approval and validation evidence, and recovery state constrain the `execution_verified` decision. An unresolved recovery state prevents this flag from

becoming true even when other evidence reports successful execution. The current mapping also keeps recovery unknown when the reported run state is missing and marks a disagreement between manifest and selected job or artifact identity as an evidence mismatch. These rules prevent omission or a top-level field from silently strengthening the composite decision. A record can therefore report backend completion while leaving the product’s recovery obligation unresolved. Export checks supplied fields and bindings, writes a separate file using exclusive creation, and leaves the engine and source run untouched. In the current mapping, `execution_verified` denotes satisfaction of the evidence policy. Missing source revision, backend-instance or validator-version metadata alone does not veto this decision.

B. Using the evidence record

For a response-loss incident, the investigator starts with the original run’s manifest. The recorded submission outcome and job identifier determine which backend evidence can be associated with that run. If submission is explicitly unknown, a later successful call belongs to its own record; its job identifier is not assigned to the original run. The investigator associates the relevant backend observation with the request and execution instance and records artifact checks separately. The export command writes to a new file, refusing to overwrite an existing export:

```
python -m
scripts.research.export_records
-input run-record.json -output
evidence.json
```

The investigator can then compare reported and interpreted state, follow the mapping reasons and inspect supplied oracle evidence. An absent oracle record appears as missing evidence in this view. Export leaves the source run intact and makes no recovery call. Windows inputs pass through a separate offline conversion step because the observer and exporter schemas differ.

The companion demonstration supplies two synthetic records with different original-run statuses. It also applies the mapping to twelve retained CPU product manifests and their fixture oracle records, with no added artifact-validation assertions. All twelve receive false composite verification. This offline exercise reproduces interpretation of the stored records; the original events are not replayed or independently certified. In a record with successful backend evidence and required recovery, the unresolved obligation still belongs to the original run, even after a later call succeeds.

C. Recovery guard and its scope

`AssetRunEngine` and `RunStore` preserve the explicit transport marker `submission_outcome=unknown` as an unresolved attempt. The reviewed submission entry checks that unresolved attempt before permitting further generation. The guard applies within the same run, including attempts with a different idempotency key. The corresponding inspection action is `get_run`.

The guard neither discovers a missing job identifier nor reconciles an unknown job. A new run lies outside its scope. Known-job recovery in two-stage operations uses a separate path. Withholding another submission leaves the outcome uncertain and supplies no completion guarantee.

A pipeline integrator can use this distinction when a generation UI receives a disconnect after sending a request: the exporter preserves the unknown outcome, the same-run entry blocks another submission, and the operator must inspect or reconcile the run through an available backend-specific mechanism. The guard can also conservatively withhold work when a request failed before backend acceptance, which is a potential false-positive direction. In the accepted-response-loss CPU fixture, the observed cost is an incomplete client operation; eventual completion is not established.

D. Controlled observation on Windows

Figure 1 shows the product and research boundaries. The research harness adds a transport shim and loopback fault proxy. The shim routes prompt transport through the proxy while retaining the actual backend endpoint in the product’s identity client. F02 drops the first response only after backend acceptance. This arrangement is a research instrument, not a native ComfyUI recovery feature.

The observer connects before the product invocation and uses the operation key as its ComfyUI client identifier, matching the product’s event-routing identity. `ComfyUIEventOracle` requires an execution-start event, a terminal event and completed history bound to a prompt identifier. It derives an execution-instance identifier using the backend boot identity, prompt identifier and observed start sequence. Its independence is from the product return value: it still trusts the backend’s event and history interfaces, and it is not a physical GPU monitor or a crash-durable audit log.

The controller permits the second F02 product call only after the first execution has a bindable completion. In this historical protocol, the client creates a new run on each call and uses `seed = 4100 + call_index`. This is observation-gated fresh-run generation, not automatic reconciliation of the original run. Because the new run is outside the same-run guard, the Windows campaign does not measure the guard’s blocking effect. CPU F02 fixtures exercise blocking; in the single-stage fixture, W3 has one execution rather than B2’s two, while leaving the operation incomplete. Matching frozen request digests does not establish full prompt equality: the inspected digest function excludes the seed and does not use its plan argument.

IV. EVIDENCE AND DEMONSTRATION

A. CPU components and deterministic fixtures

We denote the frozen baseline product revision by B2 and the revision with the same-run ambiguity guard by W3. The CPU harness separates submission events from synthetic worker-entry events. Its fixtures cover normal operation and response loss before and after job-ID acquisition, including single-stage and two-stage paths. Oracle self-checks exercise

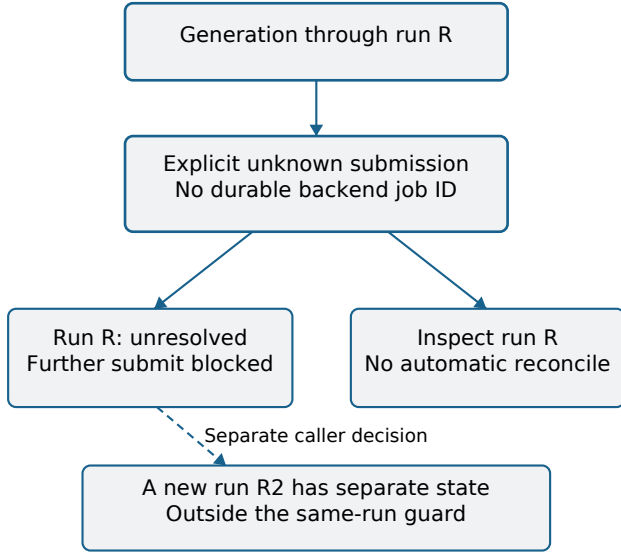


Fig. 1. Scope of the product guard, derived from the reviewed store and engine entry points. A new run has separate state. Dashed arrows denote a separate caller decision, not automatic reconciliation.

cases in which submission and execution counts differ and in which a shared prompt identifier does not imply a single execution. These are fixed semantic checks, not randomly sampled deployments.

The delivered offline checks were tested with Python 3.13.15. Their source requires Python 3.10 features; compatibility with 3.10 itself was not tested. They check retained-record consistency and mapping, not backend execution. Historical test-count and runtime claims without packaged receipts are omitted. Exact historical CPU test-worktree attestation remains unavailable. The Windows v1/v2 campaigns recorded nine harness source hashes each. Separate source snapshots reconstructed from the respective Git revisions with Windows CRLF bytes match all eighteen recorded hashes. A packaged verifier checks this correspondence; byte matching is not runtime execution attestation. The current research caller is not substituted for either execution snapshot.

Table I reports the retained paired CPU matrix. Its recovery labels are historical fixture outcomes, not exporter enumerations. In normalization v4, a `partial` reported run maps to unknown recovery, rather than no recovery requirement; this corrects the B2 F02 two-stage interpretation without changing any execution or submission count. Reanalysis checked all twelve synthetic traces: each execution-start identifier was unique within its case and matched a finish identifier, and request-received counts matched the stored submission totals. The control cases are separate from injected cases. The invalid earlier matrix version, whose single-stage control failed because of a harness result-field mismatch, is excluded.

In the F02 single-stage fixture, W3 blocks a second product submission. Its synthetic worker records one successful start and finish with an artifact, yet the product operation remains

incomplete in the observation window. B2 submits twice and completes in this fixture. The guard’s direct action is withholding a submission; the corresponding difference in synthetic execution counts is not a measured real-backend reduction. The unchanged F03 two-stage row exercises existing known-job recovery. These CPU paths must not be pooled with the Windows fresh-run protocol in Fig. 2.

B. Four controlled Windows cases

The endpoint-preserving campaign report records B2 and W3 under F00 and F02. All four fixed cases were oracle-evaluable; the two F00 cases were controls and the two F02 cases had confirmed response-loss injection. Table II reports the observations without converting these counts into a success probability.

The campaign totals were six calls, six proxy submissions and six retained bound execution instances. The F02 second call used a fresh run and changed seed. The four report records matched the corresponding retained JSONL cases. Their six retained execution bindings had distinct identifiers, completed-history flags and terminal-event labels. In both B2_F02 and W3_F02, the export reports `execution_start=3`, two proxy submissions and two retained bindings.

The export sums cumulative `raw_events` from two F02 decisions: a list containing start A followed by a list containing A and B contributes $1 + 2 = 3$ entries. Each F00 has only one decision. Distinct binding identifiers concern the two retained lifecycles, not those three entries. This mechanism explains compatibility with two bindings, but missing historical payloads and snapshot boundaries prevent excluding an additional unbound start. We report retained bindings rather than complete execution ground truth.

The first binding’s completion precedes the second receipt, as required by the controller’s observation gate; these timestamps belong to different calls. No rate is estimated. Here, “oracle-evaluable” refers to the retained binding record, not an independent raw-event audit.

The report identifies an RTX 5070 Ti Laptop GPU and ComfyUI v0.30.0 at a pinned source revision, with model and workflow hashes checked against the campaign configuration. These are recorded environment facts for this campaign. They do not establish behavior across other models, backend versions or machines.

The research export retained execution identifiers despite the first call’s ambiguous outcome. The cases provide no execution-count advantage for W3. Changing both run and seed also prevents interpreting them as repeated executions of an identical generation request.

C. Same-run scope tests and a recovered guard observation

Separate W3 same-run campaigns reused the original run and complete generation arguments after a bindable first completion. Checks covered seed, operation key, run identity and durable manifests; the capture lacks an independent retry invocation timestamp. Table III keeps these protocols separate from the fresh-run campaign and the subsequent paired study.

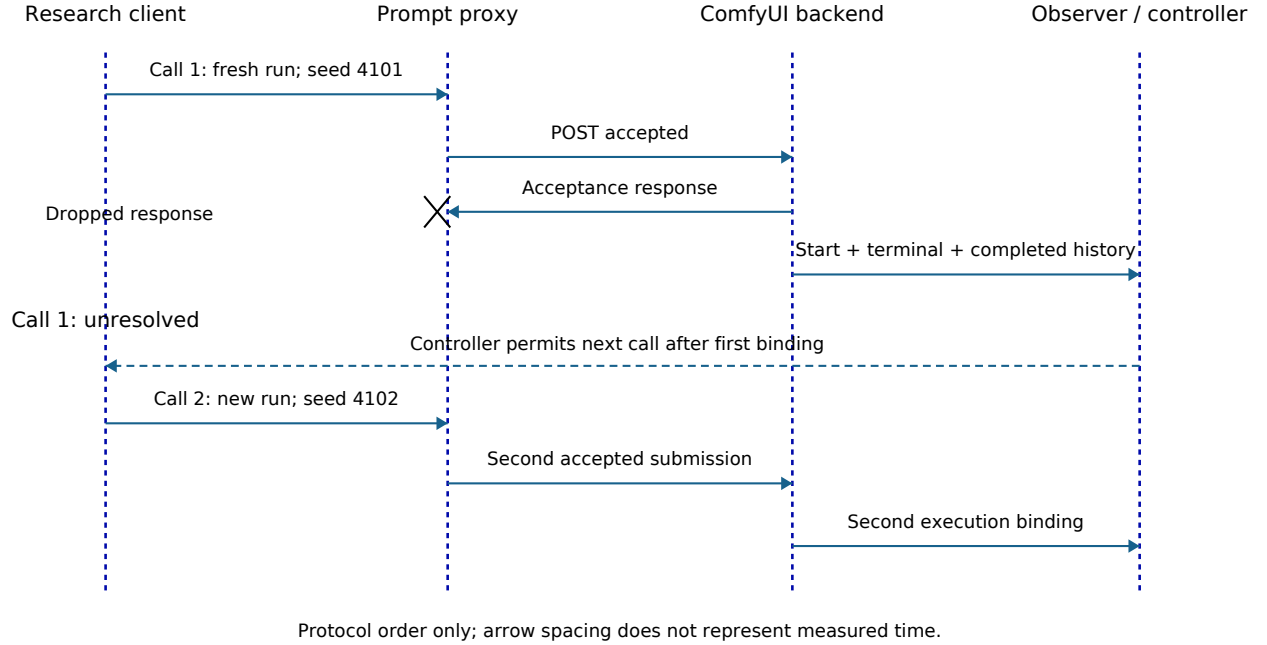


Fig. 2. Controlled F02 protocol. The proxy hides the accepted response; backend observation gates a second call that creates a fresh run with a changed seed. Retained receipts and bindings support the two submissions and the ordering of the second receipt after the first retained binding. The campaign does not exercise the same-run guard. Spacing is schematic, not a measured latency or GPU-runtime plot.

TABLE I
DETERMINISTIC CPU FAULT COVERAGE FROM THE RETAINED PAIRED-V2 MATRIX.

Fault	Path	B2 S/E	W3 S/E	Complete (B2/W3)	B2 recovery	W3 recovery
F00	Single stage	1/1	1/1	yes/yes	Not needed	Not needed
F00	Two stage	1/1	1/1	yes/yes	Not needed	Not needed
F02	Single stage	2/2	1/1	yes/no	New submission	Unknown; blocked
F02	Two stage	1/1	1/1	no/no	Unknown	Unknown; blocked
F03	Single stage	2/2	2/2	yes/yes	New submission	New submission
F03	Two stage	1/1	1/1	yes/yes	Same job reconciled	Same job reconciled

Note: S/E means submissions/executions. “Complete” is the fixture’s valid-completion flag. All twelve traces are synthetic.

TABLE II
FOUR FIXED CASES, CHECKED AGAINST RETAINED REPORT FIELDS. S/E ARE PROXY SUBMISSIONS/BOUND EXECUTION INSTANCES; U/R ARE UNRESOLVED/RESOLVED RESEARCH-CLIENT CALLS. ALL FOUR CASES WERE ORACLE-EVALUABLE.

System	Case	Calls	S/E	Call sequence
B2	F00	1	1/1	R
W3	F00	1	1/1	R
B2	F02	2	2/2	U → R
W3	F02	2	2/2	U → R

Both F00 controls completed. In v1, F02’s retry returned `model_identity_drifted`, issued no new POST and left the unresolved manifest unchanged. This rejection masked the guard; its cause remains unverified and the code alone does not establish changed model bytes. The v2 caller rebuilt the pinned inventory in both fresh MCP processes while preserving product code and locked-route validation. Its F02 retry reached `submission_outcome_unknown`, with one F02-case POST and an unchanged unresolved manifest. The guard

TABLE III
TWO CASES PER SAME-RUN WINDOWS CAMPAIGN. C/S/E DENOTE PRODUCT CALLS, PROXY SUBMISSIONS AND BOUND BACKEND LIFECYCLES. THE F02 RETRY WAS REJECTED BY MODEL-IDENTITY VALIDATION IN V1 AND BY THE SUBMISSION GUARD IN V2. IDENTICAL COUNTS ALONE CANNOT DISTINGUISH THESE MECHANISMS.

Case / protocol	C	S	E	Client calls
W3 F00 / v1	1	1	1	resolved
W3 F00 / v2	1	1	1	resolved
W3 F02 / v1	2	1	1	unresolved / unresolved
W3 F02 / v2	2	1	1	unresolved / unresolved

Note: “Client calls” contains research-client call-level labels in call order. A slash separates the first and second call. These labels are distinct from durable-run recovery states.

was observed, but the original run was not reconciled. The setup change supports a process-local inventory explanation for v1 without proving the complete internal error path.

Each campaign retained 90 F00 and 10 F02 Web-Socket payloads and one history response per case. Offline checks rehashed bytes and reconstructed start,

TABLE IV
SIX FIXED PAIRED OPERATIONS.

Case	C	S	U	A	E	Complete
B2 F00	1	1	1	1	1	yes
W3 F00	1	1	1	1	1	yes
W3 F02	2	1	1	1	1	no
B2 F02	2	2	2	2	2	yes
B2 FPPE	2	2	1	1	1	yes
W3 FPPE	2	1	0	0	0	no

C: product calls; S: proxy POSTs; U: upstream send entries; A: accepted jobs; E: bound backend lifecycles. Complete refers to the original client run. These are coverage counts, not deployment rates.

executing(`node=null`) and successful-history bindings. Both conditions included cache events. F02 cached nodes 3–9, with zero `progress` and one `progress_state` message; it does not establish new physical GPU computation. These retained bytes neither authenticate execution nor recover the missing fresh-run event lists. No third call or fresh-run fallback occurred.

D. Paired submission control and completion cost

The subsequent `paired-ambiguity-v1` campaign used the pinned native B2 and W3 products with a shared caller and matched generation semantics, including seed. Each fault retry reused its original run and complete generation arguments. F02 suppressed the first accepted response. FPPE closed the connection after body reception but before the upstream send function. The caller scheduled a retry after the first call returned, without waiting for observer completion. Each operation started a fresh backend; its two calls shared that backend cache.

Both F00 controls completed. Their generation RPC intervals, 35.77 and 24.20 seconds, set the predefined fault window to 107.32 seconds. These intervals include MCP and client processing. Table IV separates calls, proxy submissions, upstream sends, accepted jobs and bound lifecycles.

In F02, B2 retried and completed its original run. W3’s retry returned `submission_outcome_unknown`, issued no second POST and left the run unresolved despite corroborated backend completion. B2’s second lifecycle reported cached nodes 3–9, covering the generation graph. The extra lifecycle therefore provides no measurement of an additional full GPU generation.

FPPE exposes the guard’s completion cost. The proxy proved that the first request never entered the upstream send path. B2 retried and completed; W3 still withheld retry and left its run unresolved. The product guard could not use the research observer’s no-send evidence to discharge that uncertainty. This adverse case belongs alongside the F02 submission difference.

The batch completed all six operations with ten calls, eight proxy POSTs and six upstream send entries. Source bytes were frozen before backend startup. A complete private archive matched across hosts; offline reanalysis checked raw-message hashes, replayed bindings and recomputed counts and paired semantics. Two earlier attempts remain separate: a pre-start

port-check stop, and a partial batch whose observer exited on one second of message silence. The corrected observer waits through quiet intervals until the fixed deadline; the correction was tested with real sockets on both platforms before this batch.

E. Resolution labels require an explicit aggregation rule

A resolved second call does not erase the first call’s unresolved result or show that its original run was reconciled. Read-only inspection of the retained campaign report confirmed that both F02 records carry `resolved=1` and `unresolved=1`. These are independent any-call flags: across the four cases, `resolved` sums to four and `unresolved` to two. They are overlapping counts, not mutually exclusive outcomes. The earlier Markdown summary’s zero unresolved cases is therefore not the controller aggregation. Table II preserves the call sequence; the evidence addendum records this correction without rewriting the historical report. No recovery-success rate is inferred.

V. RELATED WORK AND DISCUSSION

Observation and control answer different questions at an uncertain submission boundary. A backend lifecycle can complete while the client still requires recovery; blocking a further submission does not discharge that obligation. RPC semantics and idempotent service contracts already address ambiguous outcomes and safe retries [1], [2]. Our tool instead records the evidence available when backend deduplication is unestablished. Neither a prompt identifier nor a client idempotency key alone supplies that guarantee.

Li’s preprint compares model, harness and tool-contract effects on duplicate side effects in a simulated service benchmark [3]. Its guard combines read-back, supported idempotency keys, delayed verification, blocking unverifiable repeats and uncertainty annotations. Our narrower guard blocks an explicitly unknown submission within one run, without automatic read-back or reconciliation. Here the recovery decision concerns a persistent local generation run: the export preserves its reported state and mapping reasons alongside backend observations, and further generation checks the unresolved attempt. Different workloads and evidence sources preclude comparing the studies’ outcome counts.

Reliable State Machines use persistent state and atomic runtime processing to support failure transparency and exactly-once input processing [4]. Our external generator remains outside such a transactional runtime. Temporal likewise documents that completed Activities may be retried after a worker crashes before reporting completion, and recommends idempotent Activities [5]. These contracts explain why durable client state alone cannot settle external side effects. In our pipeline, recovery of the generator’s outcome remains an application responsibility.

OpenTelemetry relates operation spans through trace context [6]. Our application-specific policy additionally requires backend start and terminal events bound to completed history,

while unresolved recovery vetoes `execution_verified`. Correlation identifiers alone are not execution counts.

Toxiproxy injects configurable TCP connection faults [7]. Our proxy selects an accepted application response for suppression while a separate observer checks its lifecycle. Filibuster’s project overview describes synthesizing fault-injected variations from integration tests [8]. Our fixed cases cover a selected submission boundary. These fault-injector comparisons use project documentation; equivalent Toxiproxy setups and Filibuster were not benchmarked.

VI. REPRODUCIBILITY AND LIMITATIONS

Offline checks use retained CPU traces, the Windows source/report ledger and sanitized projections. Live repetition requires the pinned backend, model, workflow and private setup. Historical and same-run campaigns remain separate. Export assumes trustworthy input provenance and does not authenticate sources.

Historical JSONL supports consistency checks but lacks the raw events needed to resolve the start-count discrepancy. Its constructed terminal label is weaker than the oracle’s completion rule, which requires `executing` with `node=null` and completed history. Artifact receipts cover generated-call outputs, excluding the first unresolved F02 output.

The paired cases establish submission withholding and a completion cost on one Windows workload. Desktop activity was retained as a timing confound. A simple stop-after-unknown policy matched W3’s fault submission counts in CPU checks; it was not measured on Windows. Same-information offline probes also gave the exporter and an ordinary parser identical decisions. These comparisons support an inspectable integration, rather than policy or classification superiority. The new campaign did not separately evaluate exporter normalization. The anonymous demo replays semantic mapping; four variants are synthetic, and converted F00 and F02 remain composite-unverified without an independent artifact validator. The companion evidence note details audit coverage.

VII. CONCLUSION

We built the tool to keep a recovery decision attached to the run that created the uncertainty. The paired cases show what this choice costs: W3 withheld an additional submission but left the original run unresolved, while B2 retried and completed. The pre-send fault makes the conservative block visible even when no backend job exists. Preserving these outcomes alongside backend observations gives pipeline integrators a concrete record for deciding how to reconcile the operation, rather than letting a successful retry erase its unresolved history.

DATA, CODE AND RESPONSIBLE USE

The manuscript source includes derived Windows evidence, retained CPU-record pointers, a deterministic audit script and figure sources. Models, private configuration, generated images and raw host paths are excluded from the external anonymous demo. A separate private author-review bundle

contains exact source snapshots and audit metadata with host/account identifiers and local paths; it is not an anonymous or public distribution. The analysis concerns controlled software executions; it makes no claim about human participants or image-quality acceptance. Human-subjects or institutional-ethics determinations are not inferred from this scope.

AI TOOL USAGE

OpenAI Codex assisted with drafting, author-side source/record checks and code for Figs. 1–2; Humanizer assisted with prose revision. Final content and declarations require human-author approval.

REFERENCES

- [1] A. D. Birrell and B. J. Nelson, “Implementing remote procedure calls,” *ACM Transactions on Computer Systems*, vol. 2, no. 1, pp. 39–59, 1984. [Online]. Available: <https://www.cs.cmu.edu/~15712/papers/birrell84.pdf>
- [2] M. Featonby, “Making retries safe with idempotent apis,” Amazon Builders’ Library, accessed September 27, 2026. [Online]. Available: <https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/>
- [3] J. Li, “Where does exactly-once live? model, harness, and tool-contract effects on duplicate side effects in LLM agents,” arXiv preprint arXiv:2609.29095v1, 2026, version 1, September 24, 2026. [Online]. Available: <https://arxiv.org/abs/2609.29095v1>
- [4] S. Mukherjee, N. J. Raj, K. Govindraj, P. Deligiannis, C. Ravichandran, A. Lal, A. Rastogi, and R. Krishnaswamy, “Reliable state machines: A framework for programming reliable cloud services,” in *33rd European Conference on Object-Oriented Programming (ECOOP 2019)*, ser. Leibniz International Proceedings in Informatics, vol. 134, 2019, pp. 18:1–18:29. [Online]. Available: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2019.18>
- [5] Temporal Technologies, “Activity definition,” official documentation. Accessed September 27, 2026. [Online]. Available: <https://docs.temporal.io/activity-definition>
- [6] OpenTelemetry Authors, “Traces,” official documentation. Accessed September 27, 2026. [Online]. Available: <https://opentelemetry.io/docs/concepts/signals/traces/>
- [7] Shopify, “Toxiproxy,” official project documentation. Accessed September 27, 2026. [Online]. Available: <https://github.com/Shopify/toxiproxy>
- [8] Filibuster, “Reliability starts with the application,” official project overview. Accessed September 27, 2026. [Online]. Available: <https://filibuster.cloud/>